1. OTP SYSTEM OVERVIEW

The OTP system is designed as a **secure, scalable, multi-tenant authentication service** that supports OTP delivery via **SMS and Email** using **asynchronous messaging** while keeping **OTP creation and verification synchronous**.

Key principles:

- No plaintext OTP is stored
- OTP generation uses **OS-level cryptographically secure randomness**
- OTP delivery is **decoupled from API** using a queue
- System supports **multi-tenant configuration**
- OTP is **time-bound, attempt-bound, and single-use**

---

2. HIGH-LEVEL OTP ARCHITECTURE

Logical Flow

```
UI / Mobile / Postman
        ↓
OTP API (Sync)
  - Validation
  - Rate limiting
  - OTP generation
  - Hashing (bcrypt)
  - DB write
        ↓
SQL Database (Sync)
        ↓
Message Queue (Async)
        ↓
Background Worker
        ↓
Azure Communication Service
        ↓
User (Phone / Email)
```

Sync vs Async Responsibilities

Synchronous (API Layer):

- Validate request
- Generate OTP
- Hash OTP
- Store OTP metadata
- Return response

Asynchronous (Worker Layer):

- Send OTP via SMS/Email
- Retry on failure
- Update delivery status

---

3. OTP CREATION – HOW OTP IS GENERATED

- OTP is generated using **Cryptographically Secure Pseudo-Random Number Generator (CSPRNG)** provided by the operating system

Example range for 6-digit OTP:

```
100000 – 999999
```

-
- OTP exists **only in memory for milliseconds**
- Immediately hashed using **bcrypt**
- Only the hash is stored in the database

Why this is secure:

- No mathematical RNG

- No predictable seed
- OS entropy (CPU noise, interrupts, timing)
- Resistant to prediction

---

4. OTP STORAGE & SECURITY MODEL

- OTP value is **never stored**
- Only `bcrypt` hash is stored
- OTP verification uses `bcrypt.compare()`
- OTP is invalidated after:
    - Successful verification
    - Expiry
    - Max attempts exceeded
    - Resend request

Security controls:

- Rate limiting per tenant + identifier
- Max verification attempts
- One-time use
- Generic error messages (no enumeration)

---

5. OTP VERIFICATION FLOW

1. User submits OTP
2. API fetches latest active OTP record

3. Validations:
    - Not expired
    - Not already verified
    - Attempts < max
4. Compare using `bcrypt.compare(inputOtp, storedHash)`
5. On success:
    - Mark OTP as VERIFIED
6. On failure:
    - Increment attempt count
    - Lock after max attempts

---

6. DATABASE DESIGN (CORE TABLES)

6.1 OTP Table

Table: `Otps`

Column	Type	Constraints	Description
Id	UNIQUEIDENTIFIER	PK	OTP ID
TenantId	UNIQUEIDENTIFIER	FK	Tenant reference
Identifier	NVARCHAR(255)	NOT NULL	Email / Phone
Channel	NVARCHAR(10)	NOT NULL	SMS / EMAIL
Purpose	NVARCHAR(50)	NOT NULL	LOGIN / SIGNUP / RESET
OtpHash	NVARCHAR(255)	NOT NULL	bcrypt hash
Status	NVARCHAR(20)	NOT NULL	PENDING / SENT / VERIFIED / EXPIRED

Attempts	INT	DEFAULT 0	Attempt count
ExpiresAt	DATETIME2	NOT NULL	Expiry
CreatedAt	DATETIME2	DEFAULT SYSUTCDATETIME()	Created time
VerifiedAt	DATETIME2	NULL	Verification time

6.2 OTP Audit Logs

Table: `OtpAuditLogs`

Column	Type	Description
Id	UNIQUEIDENTIFIER	Log ID
OtpId	UNIQUEIDENTIFIER	OTP reference
Action	NVARCHAR(50)	OTP_CREATED / SENT / VERIFIED
Metadata	NVARCHAR(MAX)	JSON
CreatedAt	DATETIME2	Timestamp

---

7. API ENDPOINTS (SUMMARY)

Method	Endpoint	Purpose
POST	`/api/v1/otp/request`	Generate OTP
POST	`/api/v1/otp/verify`	Verify OTP
POST	`/api/v1/otp/resend`	Resend OTP
GET	`/api/v1/otp/status/{otpId}`	OTP status
GET	`/api/v1/admin/otp/logs`	Audit logs
GET	`/api/v1/tenants`	Tenant list

---

8. JSON FORMATS (FINAL)

8.1 Request OTP

POST `/otp/request`

```
{
  "identifier": "user@example.com",
  "channel": "EMAIL",
  "purpose": "LOGIN"
}
```

Response

```
{
  "success": true,
  "data": {
    "otpId": "uuid",
    "expiresInSeconds": 300
  }
}
```

8.2 Verify OTP

POST `/otp/verify`

```
{
  "identifier": "user@example.com",
  "otp": "123456",
  "purpose": "LOGIN"
}
```

Response

```
{
  "success": true,
  "data": {
    "verified": true
  }
}
```

8.3 Resend OTP

POST `/otp/resend`

```
{
  "identifier": "user@example.com",
  "channel": "EMAIL",
  "purpose": "LOGIN"
}
```

8.4 OTP Status

GET `/otp/status/{otpId}`

```
{
  "success": true,
  "data": {
    "status": "SENT",
    "attempts": 1,
    "expiresAt": "2026-01-07T12:40:00Z"
  }
}
```

---

9. EDGE CASES HANDLED

- Multiple OTP requests → previous OTP invalidated
- Expired OTP → rejected
- Too many attempts → locked
- SMS/Email failure → async retry
- Worker crash → message reprocessed
- Enumeration prevention → generic errors

---

10. FINAL ONE-LINE SUMMARY (EXECUTIVE)

> This OTP system uses OS-level secure randomness, bcrypt hashing, asynchronous delivery, and strict rate-limiting to provide a scalable, multi-tenant, audit-ready authentication mechanism without ever storing plaintext OTPs.
